Activity 1.16

1 A camera detector has an array of 1920 by 1536 pixels. A colour depth of 16 bits is used. Calculate the size of a photograph taken by this camera, giving your answer in MiB.

2 Photographs have been taken by a smartphone which uses a detector with a 1024 × 1536 pixel array. The software uses a colour depth of 24 bits. How many photographs could be stored on a 16 GiB memory card?

3 Audio is being sampled at the rate of 44.1 kHz using 8 bits. Two channels are being used. Calculate:

   a the size of a one second sample, in bits

   b the size of a 30-second audio recording in MiB.

4 The typical song stored on a music CD is 3 minutes and 30 seconds. Assuming each song is sampled at 44.1 kHz (44 100 samples per second) and 16 bits are used per sample. Each song utilises two channels.

   Calculate how many typical songs could be stored on a 740 MiB CD.

1.3.3 Data compression

The calculations in Section 1.3.2 show that sound and image files can be very large. It is therefore necessary to reduce (or **compress**) the size of a file for the following reasons:

- to save storage space on devices such as the hard disk drive/solid state drive
- to reduce the time taken to stream a music or video file
- to reduce the time taken to upload, download or transfer a file across a network
- the download/upload process uses up network **bandwidth** – this is the maximum rate of transfer of data across a network, measured in bits per second. This occurs whenever a file is downloaded, for example, from a server. Compressed files contain fewer bits of data than uncompressed files and therefore use less bandwidth, which results in a faster data transfer rate.
- reduced file size also reduces costs. For example, when using cloud storage, the cost is based on the size of the files stored. Also an internet service provider (ISP) may charge a user based on the amount of data downloaded.

1.3.4 Lossy and lossless file compression

File compression can either be **lossless** or **lossy**.

Lossy file compression

With this technique, the file compression algorithm eliminates unnecessary data from the file. This means the original file cannot be reconstructed once it has been compressed.

Lossy file compression results in some loss of detail when compared to the original file. The algorithms used in the lossy technique have to decide which parts of the file need to be retained and which parts can be discarded.

For example, when applying a lossy file compression algorithm to:

- an image, it may reduce the resolution and/or the bit/colour depth
- a sound file, it may reduce the sampling rate and/or the resolution.

Lossy files are smaller than lossless files which is of great benefit when considering storage and data transfer rate requirements.

Common lossy file compression algorithms are:

- MPEG-3 (MP3) and MPEG-4 (MP4)
- JPEG.

MPEG-3 (MP3) and MPEG-4 (MP4)

MP3 files are used for playing music on computers or mobile phones. This compression technology will reduce the size of a normal music file by about 90%. While MP3 music files can never match the sound quality found on a DVD or CD, the quality is satisfactory for most general purposes.

But how can the original music file be reduced by 90% while still retaining most of the music quality? Essentially the algorithm removes sounds that the human ear can't hear properly. For example:

- removal of sounds outside the human ear range
- if two sounds are played at the same time, only the louder one can be heard by the ear, so the softer sound is eliminated. This is called perceptual music shaping.

MP4 files are slightly different to MP3 files. This format allows the storage of multimedia files rather than just sound – music, videos, photos and animation can all be stored in the MP4 format. As with MP3, this is a lossy file compression format, but it still retains an acceptable quality of sound and video. Movies, for example, could be streamed over the internet using the MP4 format without losing any real discernible quality.

JPEG

When a camera takes a photograph, it produces a raw bitmap file which can be very large in size. These files are temporary in nature. **JPEG** is a lossy file compression algorithm used for bitmap images. As with MP3, once the image is subjected to the JPEG compression algorithm, a new file is formed and the original file can no longer be constructed.

The JPEG file reduction process is based on two key concepts:

- human eyes don't detect differences in colour shades quite as well as they detect differences in image brightness (the eye is less sensitive to colour variations than it is to variations in brightness)
- by separating pixel colour from brightness, images can be split into 8 × 8 pixel blocks, for example, which then allows certain 'information' to be discarded from the image without causing any real noticeable deterioration in quality.

Lossless file compression

With this technique, all the data from the original uncompressed file can be reconstructed. This is particularly important for files where any loss of data would be disastrous (e.g. when transferring a large and complex spreadsheet or when downloading a large computer application).

Lossless file compression is designed so that none of the original detail from the file is lost.

Run-length encoding (RLE) can be used for lossless compression of a number of different file formats:

- it is a form of lossless/reversible file compression
- it reduces the size of a string of adjacent, identical data (e.g. repeated colours in an image)
- a repeating string is encoded into two values:
  - the first value represents the number of identical data items (e.g. characters) in the run
  - the second value represents the code of the data item (such as ASCII code if it is a keyboard character)
- RLE is only effective where there is a long run of repeated units/bits.

Using RLE on text data

Consider the following text string: 'aaaaabbbbccddddd'. Assuming each character requires 1 byte then this string needs 16 bytes. If we assume ASCII code is being used, then the string can be coded as follows:

a	a	a	a	A	b	b	b	b	c	c	d	d	d	d	d
05 97					04 98				02 99		05 100				

This means we have five characters with ASCII code 97, four characters with ASCII code 98, two characters with ASCII code 99 and five characters with ASCII code 100. Assuming each number in the second row requires 1 byte of memory, the RLE code will need 8 bytes. This is half the original file size.

One issue occurs with a string such as 'cdcdcdcdcd' where RLE compression isn't very effective. To cope with this, we use a flag. A flag preceding data indicates that what follows are the number of repeating units (for example, 255 05 97 where 255 is the flag and the other two numbers indicate that there are five items with ASCII code 97). When a flag is not used, the next byte(s) are taken with their face value and a run of 1 (for example, 01 99 means one character with ASCII code 99 follows).

Consider this example:

String	aaaaaaaa	bbbbbbbbbb	c	d	c	d	c	d	eeeeeeee
Code	08 97	10 98	01 99	01 100	01 99	01 100	01 99	01 100	08 101

The original string contains 32 characters and would occupy 32 bytes of storage.

The coded version contains 18 values and would require 18 bytes of storage.

Introducing a flag (255 in this case) produces:

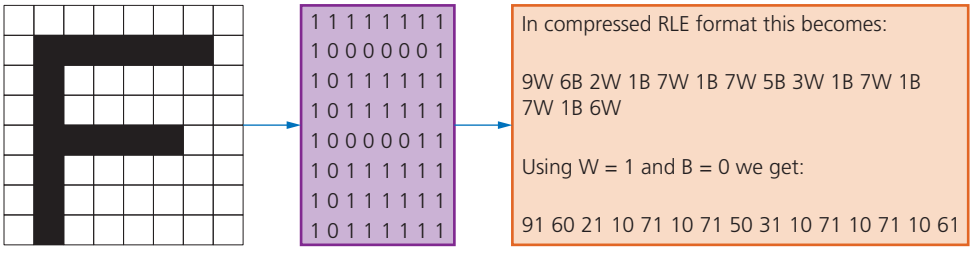
```
255 08 97 || 255 10 98 || 99 100 99 100 99 100 || 255 08 101
```

This has 15 values and would, therefore, require 15 bytes of storage. This is a reduction in file size of about 53% when compared to the original string.

Using RLE with images

Example 1: Black and white image

Figure 1.12 shows the letter 'F' in a grid where each square requires 1 byte of storage. A white square has a value 1 and a black square a value of 0:

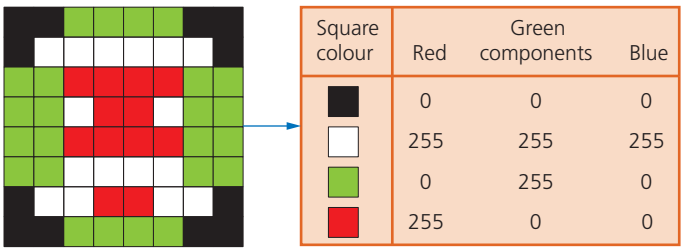


▲ **Figure 1.12** Using RLE with a black and white image

The 8 × 8 grid would need 64 bytes; the compressed RLE format has 30 values, and therefore needs only 30 bytes to store the image.

Example 2: Coloured images

Figure 1.13 shows an object in four colours. Each colour is made up of red, green and blue (RGB) according to the code on the right.



Square colour	Red	Green components	Blue
	0	0	0
	255	255	255
	0	255	0
	255	0	0

▲ **Figure 1.13** Using RLE with a coloured image

This produces the following data: 2 0 0 0 4 0 255 0 3 0 0 0 6 255 255 255 1 0 0 0 2 0 255 0 4 255 0 0 4 0 255 0 1 255 255 255 2 255 0 0 1 255 255 255 4 0 255 0 4 255 0 0 4 0 255 0 4 255 255 255 2 0 255 0 1 0 0 0 2 255 255 255 2 255 0 0 2 255 255 255 3 0 0 0 4 0 255 0 2 0 0 0.

The original image (8 × 8 square) would need 3 bytes per square (to include all three RGB values). Therefore, the uncompressed file for this image is 8 × 8 × 3 = 192 bytes.

The RLE code has 92 values, which means the compressed file will be 92 bytes in size. This gives a file reduction of about 52%. It should be noted that the file reductions in reality will not be as large as this due to other data which needs to be stored with the compressed file (e.g. a file header).